



PSW Dispersion Simulation

병렬화 최적화 · 기술 참조 문서

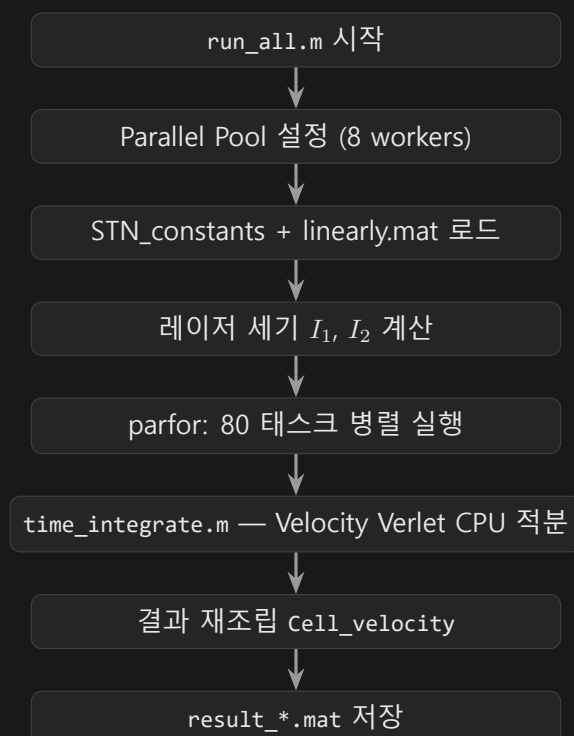
2026년 4월 · i7-13700K / RTX 4060 Ti / DDR5 64 GB

PSW(Polarized Spin-Wave) Dispersion Simulation 전체 문서 허브. `run_all.m`을 기준으로 각 모듈별 설명으로 구성된다. 이미지 처리(image processing) 관련 내용은 **PSW 이미지 처리 분석** 별도 문서로 분리되어 있다.

문서 구성

페이지	내용
1. 개요 및 설정	병렬 전략, Parallel Pool, 상수/파라미터 설정
2. 시뮬레이션 핵심 로직	linearly.mat 로드, 레이저 세기 계산, parfor sub-batch 루프
3. 후처리 및 결과 저장	결과 재조립, .mat 저장
2-1. time_integrate.m 분석	Velocity Verlet 적분, GPU/CPU 경로, axay_opt_partial 호출 구조

전체 실행 흐름



⚙️ 1. 개요 및 설정

🔗 run_all.m의 Section 0-3을 다룬다: Parallel Pool 설정, STN 상수 로드, 레이저 세기 파라미터 계산.

Section 0 — Parallel Pool 설정

MATLAB local profile의 MaxNumWorkers 기본값은 물리 P-core 수(8)로 제한된다. parcluster API를 통해 직접 프로파일을 수정해야 원하는 worker 수를 사용할 수 있다.

```
1 n_workers = 8; % 실측 최적값
2
3 p = gcp('nocreate');
4 if ~isempty(p) && p.NumWorkers ~= n_workers
5     delete(p); p = [];
6 end
7 if isempty(p)
8     cluster = parcluster('local');
9     if cluster.NumWorkers ~= n_workers
10         cluster.NumWorkers = n_workers;
11         saveProfile(cluster); % 프로파일에 영구 저장
12     end
13     parpool(cluster, n_workers);
14     fprintf('Parallel pool 시작: %d workers\n', n_workers);
15 end
```

Section 1 — STN 상수 로드 및 MAX 오버라이드

```
1 load(fullfile(root_dir, 'STN_constants.mat'));
2 MAX = 2000000; % 의 STN_constants MAX=20,000 -> 200,000 오버라이드
```

💡 STN_constants.mat에는 MAX=20000이 저장되어 있으나 시뮬레이션은 200,000 입자가 필요하므로 로드 직후 오버라이드한다.

Section 3 — 양자상태 파일 경로

```
1 jm_data_path = fullfile(root_dir, 'data', jm_data);
2 if ~isfile(jm_data_path)
3     error('파일 없음: %s', jm_data_path);
4 end
```

🏗️ 2. 시뮬레이션 핵심 로직

🔗 run_all.m의 Section 4-6을 다룬다: linearly.mat 로드, 레이저 세기 계산, parfor sub-batch 루프.

Section 4 — linearly.mat 로드 및 2D reshape

```
1 load(fullfile(root_dir, 'linearly.mat'));
2 [nj_lin, nm_lin, ~] = size(linearly);
3 linearly_2d = reshape(linearly, nj_lin*nm_lin, []);
4 % (nj*nm) x nI -- 모든 parfor 에 worker broadcast
5 clear linearly;
```

💡 linearly.mat는 $\langle \cos^2 \theta \rangle$ 록업 테이블이다. reshape으로 2D로 펼친 뒤 broadcast 변수로 사전 로드하면, 각 worker가 매 task마다 파일을 다시 읽는 오버헤드를 없앤다.

Section 5 — 레이저 세기 사전 계산

```

1 max_I = 2.0 / (pi * w0z * w0y) * sqrt(4.0 * log(2) / (pi * tau^2));
2 max_I = max_I * atten1 * atten2 * 1e-3;
3 I1_val = max_I * I1_list; % 스칼라
4 I2_vals = max_I * I2_list(:); % n_groups x 1

```

Section 6 — parfor Sub-Batch 병렬 루프

Sub-batch 분해 전략

$$\underbrace{200,000}_{\text{MAX/조건}} \times \underbrace{8}_{n_groups} \longrightarrow \underbrace{20,000}_{\text{batch_size}} \times \underbrace{80}_{n_tasks}$$

각 태스크 번호 $\text{task} \in [1, 80]$ 에서 조건 인덱스 매핑:

$$g = \left\lceil \frac{\text{task}}{n_{\text{sub}}} \right\rceil$$

```

1 batch_size = 20000;
2 n_sub      = ceil(MAX / batch_size); % 10
3 n_tasks    = n_groups * n_sub;      % 80
4 results    = cell(n_tasks, 1);
5
6 parfor task = 1:n_tasks
7     g_idx = ceil(task / n_sub);
8
9     [particle_t, laser_t] = init_particles(jm_data_path, batch_size, ...
10        x_position_fwhm, Dye_sigma, Dye_tau, ...
11        vx_caused_tilted, x_init_vel_sigma, y_init_vel_sigma, ...
12        z_init_vel_off, z_init_vel_sigma, Ij_distribution, Ips_z, Ips_y);
13
14     j_idx_t = particle_t(:,10) + 1;
15     m_idx_t = particle_t(:,11) + 1;
16     aligned_cost_t = linearly_2d(sub2ind([nj_lin,nm_lin], j_idx_t,m_idx_t), :);
17
18     I1_t = repmat(I1_val, batch_size, 1);
19     I2_t = repmat(I2_vals(g_idx), batch_size, 1);
20
21     results{task} = time_integrate( ...
22         particle_t(:,1:3), particle_t(:,4:6), ...
23         laser_t(:,3), laser_t(:,4:5), ...
24         I1_t, I2_t, aligned_cost_t, zeros(batch_size,2), ...
25         lambda, w0z, w0y, tau, batch_size, tstep, t_ini, tend, false);
26 end

```

3. 후처리 및 결과 저장

✧ run_all.m의 후처리 부분(Section 7, 9)을 다룬다. sub-batch 결과 재조립 → 결과 저장 순서로 진행된다. 이미지 처리(Section 8)는 PSW 이미지 처리 분석 문서로 분리됨.

Section 7 — Sub-batch 결과 재조립

80개 태스크의 결과를 조건(I_2 세기)별로 합쳐 Cell_velocity에 저장한다.

```

1 Cell_velocity = cell(n_groups, 1);
2 for g = 1:n_groups
3     task_idx = (g-1)*n_sub + (1:n_sub);
4     Cell_velocity{g} = vertcat(results{task_idx}); % MAX x 3
5 end

```

💡 `vertcat(results{task_idx})`는 셀 배열 슬라이싱과 수직 결합을 한 번에 처리한다. 각 cell은 `batch_size × 3` 행렬이므로 결합 후 `MAX × 3` 행렬이 된다.

Section 9 — 결과 저장

결과 파일명은 `jm_data` 파일명 기반으로 자동 생성된다. 저장 전에 `constants` 구조체를 먼저 구성한다.

`constants` 구조체 구성

```

1 % 시뮬레이션 제어
2 constants.n_workers      = n_workers;
3 constants.MAX            = MAX;
4 constants.batch_size     = batch_size;
5 % 이미지 처리 파라미터 (-> PSW 이미지 처리 분석 문서 참조)
6 constants.rep_std       = rep_std;
7 constants.rep_vel       = rep_vel;
8 constants.vel_pix       = vel_pix;
9 constants.atten_factorx = atten_factorx;
10 constants.atten_factory = atten_factory;
11 % 분자빔 초기조건
12 constants.jm_data       = jm_data;
13 constants.x_position_fwhm = x_position_fwhm;
14 constants.Dye_sigma     = Dye_sigma;
15 constants.Dye_tau       = Dye_tau;
16 constants.vx_caused_tilted = vx_caused_tilted;
17 constants.x_init_vel_sigma = x_init_vel_sigma;
18 constants.y_init_vel_sigma = y_init_vel_sigma;
19 constants.z_init_vel_off  = z_init_vel_off;
20 constants.z_init_vel_sigma = z_init_vel_sigma;
21 constants.Ij_distribution = Ij_distribution;
22 constants.Ips_z          = Ips_z;
23 constants.Ips_y          = Ips_y;
24 % 레이저 초기조건
25 constants.lambda = lambda;
26 constants.w0z    = w0z;
27 constants.w0y    = w0y;
28 constants.tau    = tau;
29 constants.atten1 = atten1;
30 constants.atten2 = atten2;
31 constants.tstep  = tstep;
32 constants.t_ini  = t_ini;
33 constants.tend   = tend;

```

저장 명령

```

1 out_file = fullfile(root_dir, ...
2     sprintf('result_%s.mat', strrep(jm_data, '.txt', '')));
3 save(out_file, 'Cell_velocity', 'I1_list', 'I2_list', 'constants');
4 fprintf('저장 완료: %s\n', out_file);

```

저장 변수 목록

변수	설명
Cell_velocity	조건별 최종 속도 데이터 (n_groups×1 cell, 각 MAX×3)
I1_list, I2_list	사용된 레이저 세기 목록
constants	시뮬레이션 전체 파라미터 구조체 — 제어(n_workers, MAX, batch_size), 분자빔 초기조건, 레이저 초기조건, 이미지 처리 파라미터 포함

성능 벤치마크

방식	소요 시간	배속
순차 실행	~960 s	1×
parfor 8 workers	~181 s	5.3×
parfor 12 workers	~183 s	5.2×
parfor 16 workers	~187 s	5.1×
parfor 24 workers	~208 s	4.6×

⚡ 8 workers가 최적값이다. Worker 수를 늘릴수록 aligned_cost 배열(broadcast 변수)의 동시 접근이 DDR5 메모리 버스를 포화시켜 오히려 성능이 저하된다.

⚡ 2-1. time_integrate.m 분석

🔗 time_integrate.m은 Velocity Verlet 시간 적분기로, GPU/CPU 두 경로를 모두 지원한다.

함수 시그니처

```
1 function velocity = time_integrate(position, velocity, Ij, Ips, ...
2     I1_vec, I2_vec, aligned_cost, last_data, ...
3     lambda, w0z, w0y, tau, MAX, tstep, t_ini, tend, use_gpu)
4 % use_gpu: true -> GPU 사용 단독(실행 시)
5 %         false -> CPU 사용 (parfor worker 내 권장)
6 %         생략 -> GPU 자동 감지
```

CPU 경로 (parfor worker 내부)

```
1 t = t_ini;
2 while t < tend
3     position = position + velocity .* tstep;
4     [~, last_data, a, ~] = axay_opt_partial( ...
5         position, t + Ij, I1_vec, I2_vec, ...
6         aligned_cost, last_data, Ips, lambda, w0z, w0y, tau, MAX);
7     velocity = velocity + a .* tstep;
8     t = t + tstep;
9 end
```

💡 총 6,000 스텝 ($t_{ini} = -30 \text{ ns}$, $t_{end} = +30 \text{ ns}$, $\Delta t = 10 \text{ ps}$). 각 스텝에서 위치 업데이트 → 광학 힘 계산 → 속도 업데이트 순서로 진행된다.

GPU 경로 (단독 실행 시)

```
1 g_dev = gpuDevice;
2 nI = size(aligned_cost, 2);
3 bytes_per_particle = nI * 8 + 250;
4 gpu_batch_size = floor(g_dev.AvailableMemory * 0.8 / bytes_per_particle);
5 gpu_batch_size = min(gpu_batch_size, MAX);
6 % VRAM 가용 메모리 기반 자동 청킹
```

⚠️ 현재 parfor worker 내부에서는 use_gpu=false로 CPU 모드 강제 사용. 20,000 입자 규모에서는 GPU kernel launch overhead가 이득을 상쇄한다 (실측: GPU 84 s vs CPU 64 s).

🔑 병렬화 전략 상세

MaxNumWorkers 강제 확장

MATLAB local profile은 MaxNumWorkers 기본값이 8이다. 단순히 parpool(N)만 호출하면 8개 이상 연결되지 않는다.

```
1 cluster = parcluster('local');
2 if cluster.NumWorkers ~= n_workers
3     cluster.NumWorkers = n_workers;
4     saveProfile(cluster); % 프로파일에 영구 저장
5 end
6 parpool(cluster, n_workers);
```

핵심 최적화 요약

최적화 항목	내용
aligned_cost 벡터화	for 루프(MAX회) → reshape+sub2ind 단일 연산으로 대체
gauss_t 중복 제거	시간 루프 내 6회 중복 계산 $\exp(-4\ln 2 \cdot t^2/\tau^2) \rightarrow$ 1회 사전 계산
broadcast 변수	linearly_2d를 parfor 외부에서 한 번 로드, 모든 worker에 공유
col_idx 경계 처리	$\min(I_floor+1, n_cols-1)$ 로 룩업 테이블 out-of-range 방어
GPU 호환 할당	zeros(MAX,3,'like',position) — CPU/GPU 경로 공통 코드 유지

알려진 한계 및 개선 방안

항목	내용
RAM 사용률	~85% (64 GB 기준 ~54 GB): batch_size 증가 불가
메모리 대역폭	8 workers가 포화점 → 추가 병렬화 불가
Single precision 전환	linearly_2d + aligned_cost를 single로 변환 시 메모리 85% → 55%, 속도 10~15% 향상 예상
GPU 재검토	입자 수 > 100,000/batch 규모에서 재평가 가능

이미지 처리(전자 반동 노이즈, 2D Voigt PSF, hist3 합성곱, 프로젝트션)는 **PSW 이미지 처리 분석** 문서를 참조.